



DEBRE BERHAN UNIVERSITY

College of Computing

Chapter 5: Database Programming

Modern Desktop Development Using API and EF Core (Code-First)

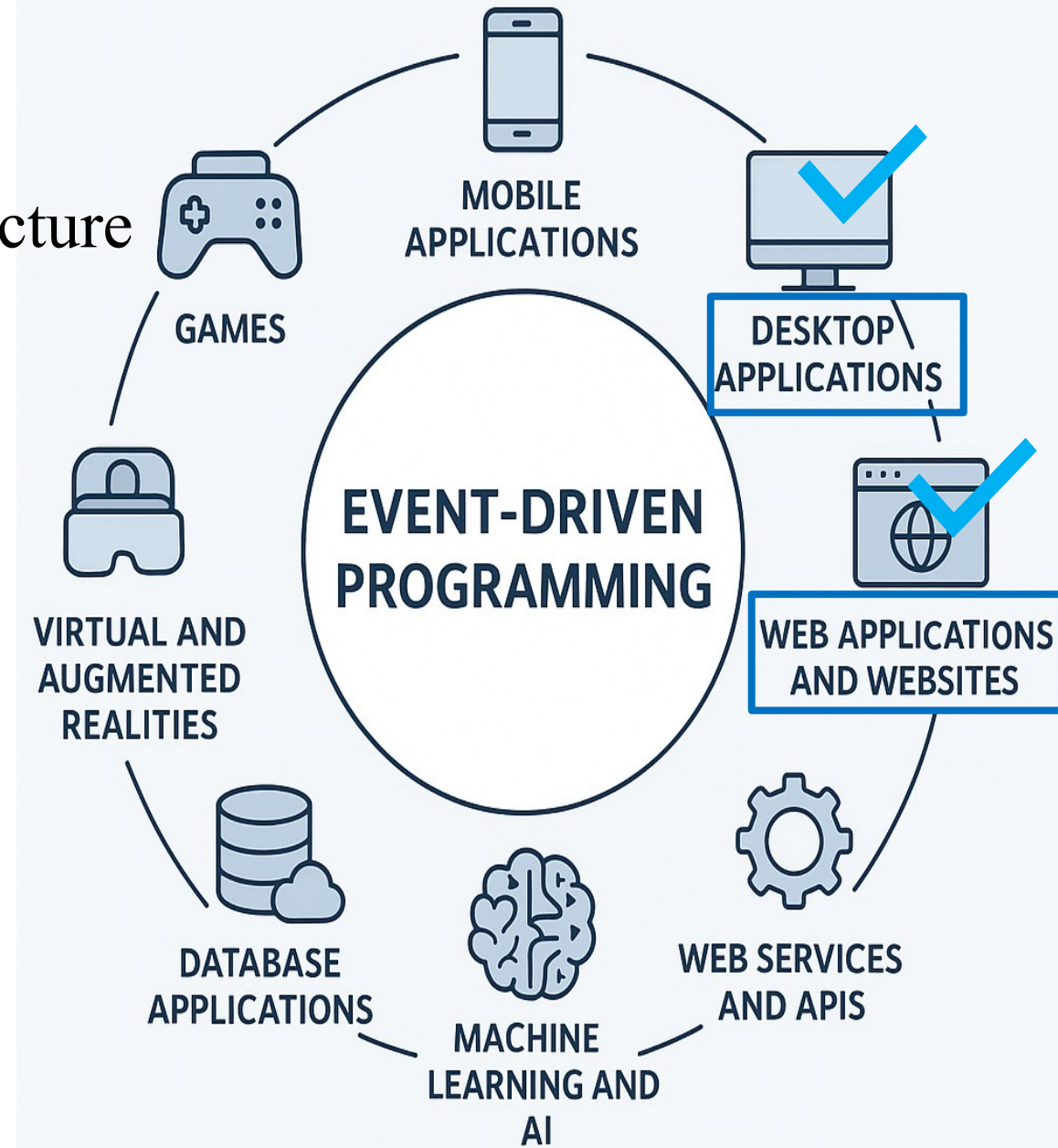
By Binyam Tekalign

August 13, 2026



Outline

- Introduction to Modern Desktop Architecture
- Data Layer with EF Core (Code-First)
- Web API Layer (ASP.NET Core)
- Desktop App Layer (WinForms)
- LINQ Essentials for EF Core



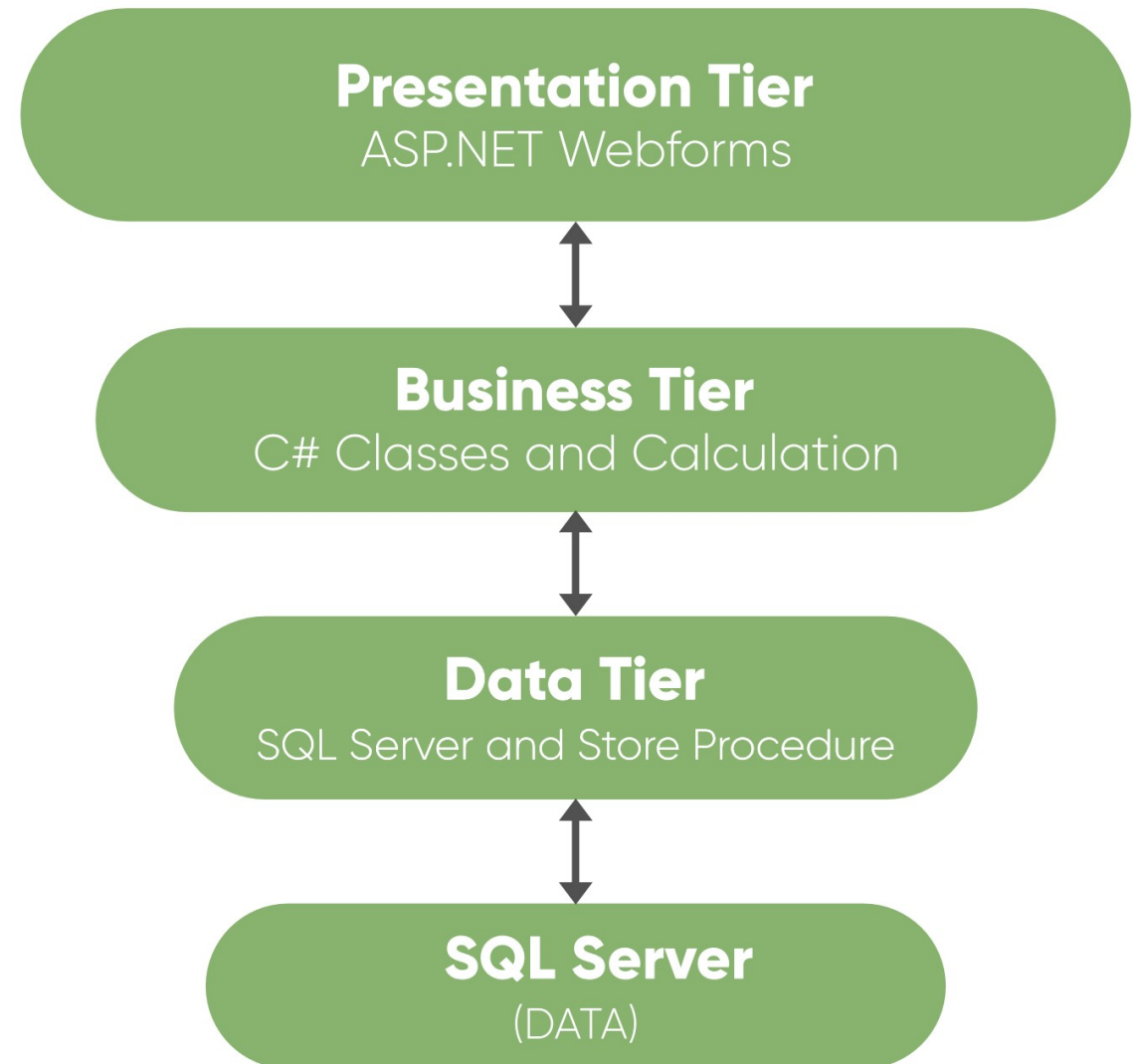
Introduction

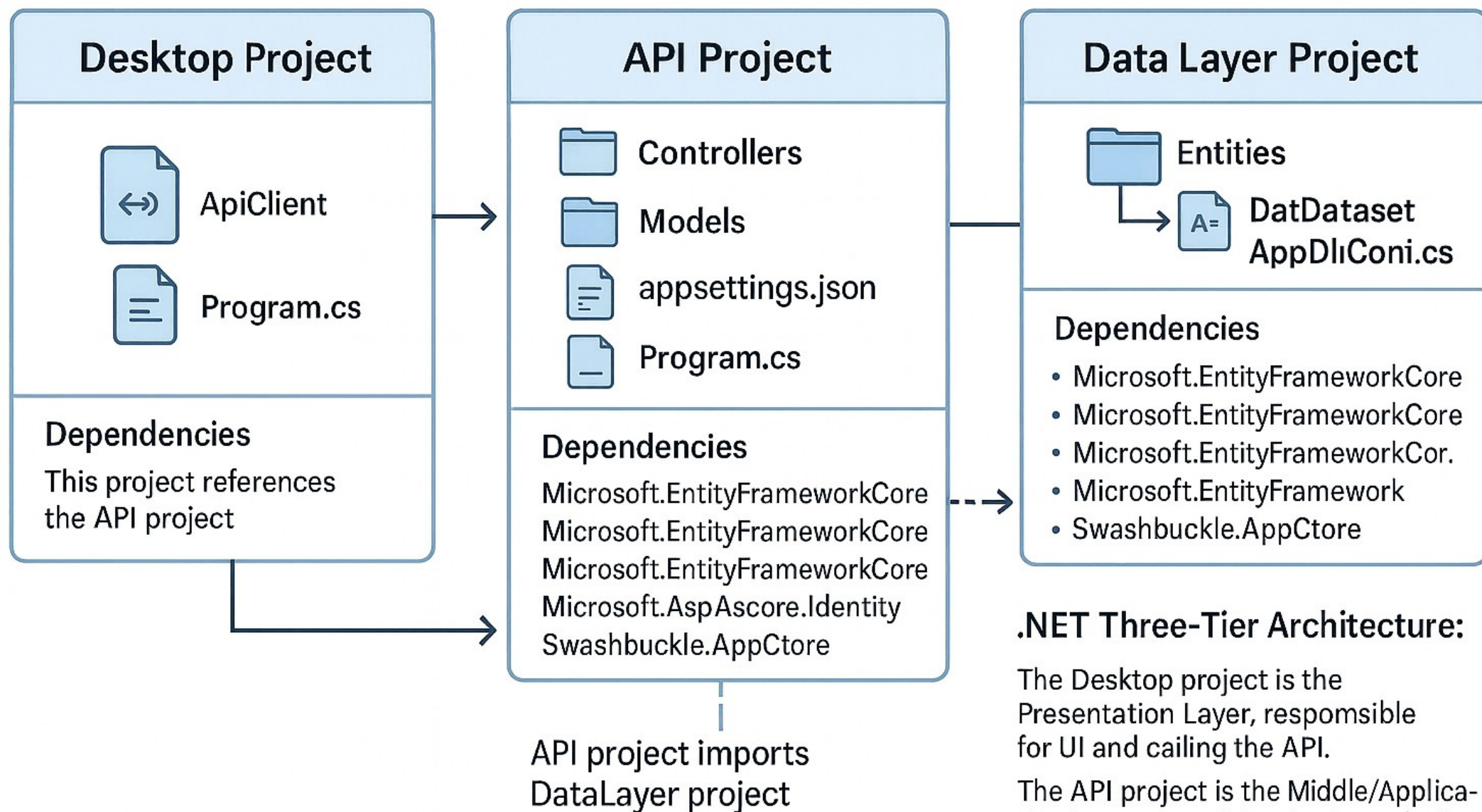
- How modern software works:
 - No more direct SQL queries in the UI
 - No more DataSet, DataAdapter, SqlCommand
 - Instead: clean separation
 - Desktop App (UI)
 - API (Logic + Services)
 - Data Layer (EF Core + DbContext)
 - You can extend this to Web, Mobile, Cloud later

What We Will Build

- A complete small system:
- **User Authentication**
 - Register user
 - Login
 - Change Password
- **Product Management (CRUD)**
 - Create
 - Read
 - Update
 - Delete
- Using:
 - **Desktop UI**
 - **Web API**
 - **Entity Framework Core**
 - **SQL Server**
 - **Code-First Migrations**

.Net Three Tier Architecture





.NET Three-Tier Architecture:

The Desktop project is the Presentation Layer, responsible for UI and calling the API.

The API project is the Middle/Application Layer, containing business logic and controllers.

The Data Layer contains database access code, EP Core entities, and AppDbContext.

Three-Project Architecture

- We will create **three separate projects**:
- **Inventory.Data**
 - Entities (Product, User...)
 - DbContext
 - EF Core
 - Migrations
- **Inventory.Api (Web API)**
 - Controllers
 - Endpoints
 - Connects to DbContext
- **Inventory.Desktop (WinForms)**
 - Forms
 - Buttons, events
 - Calls API using HttpClient
- This is the modern, clean structure.

Why Three Projects?

- **1. Separation of Responsibilities**
 - UI should not know SQL
 - API should not know UI
 - Data layer should not know how UI looks
- **2. Reusability**
 - Same API can be used by Desktop and Web apps
 - Future mobile apps can reuse the API
 - Future Blazor or MVC apps reuse everything
- **3. Maintainability**
 - Changes in database or API do not break your UI.

Traditional ADO.NET vs Modern Approach

Old Method	Modern Method
SqlConnection	HttpClient
SqlCommand	API Endpoint
DataAdapter	EF Core
DataSet	JSON Models
DB logic mixed with UI	Clean separation
Hard to share DB	API usable by any app

Overview of Inventory.Data Project

- **Class Library Project**
- **Purpose:**
 - Holds everything related to **data and database**
 - Contains **Entity classes** (tables)
 - Contains **DbContext**
 - Contains **EF Core configuration**
 - Contains **Migrations**
- **This project is shared by:**
 - The API
 - The Desktop App

NuGet Packages for Inventory.Data

■ **Microsoft.EntityFrameworkCore**

- Core EF library
- Enables entity mapping and LINQ queries

■ **Microsoft.EntityFrameworkCore.SqlServer**

- EF provider for SQL Server
- Converts C# queries → T-SQL

■ **Microsoft.EntityFrameworkCore.Tools**

- Needed for:
 - Add-Migration
 - Update-Database
- Allows CLI and PMC migrations

■ **Microsoft.EntityFrameworkCore.Design**

- Helps Visual Studio generate migration code

What Is EF Core

- **EF Core = Object-Relational Mapper (ORM)**
- **Instead of writing SQL manually:**
 - You create C# classes
 - EF creates tables automatically
 - EF performs CRUD operations
- **EF Core uses:**
 - Entities
 - DbContext
 - Migrations

Code-First Approach

- **Code-First means:**
 - You write C# classes first
 - EF Core generates the database later
- **Class example:**
- **EF creates:**
- **CREATE TABLE Products (...);**
- **Benefits:**
 - No SQL writing
 - Less errors
 - Works great with migrations
 - Easy to modify structure

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Creating the Product Entity

- Keep Points
 - Properties = table columns
 - EF identifies Id as primary key
 - Decimal = money type

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = null!;
    public decimal UnitPrice { get; set; }
    public int QuantityInStock { get; set; }
}
```

Creating the User Entity

- This table supports:
 - Register
 - Login
 - Change Password
 - We store **PasswordHash** instead of plain password.

```
public class AppUser
{
    public int Id { get; set; }
    public string UserName { get; set; } = null!;
    public string PasswordHash { get; set; } = null!;
}
```

DbContext Overview

- AppDbContext is the **heart of EF Core**
- Connects to SQL Server
- Represents tables using DbSet<T>
- Builds the model
- Manages queries and save changes
- The API will use this to access the database.

DbContext Example

- Key points:
 - DbSet<Product> = Products table
 - DbSet<AppUser> = Users table
 - The API injects this DbContext to work with SQL Server

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Product> Products => Set<Product>();
    public DbSet<AppUser> Users => Set<AppUser>();
}
```

What Is OnModelCreating?

- This method configures:

- Relationships
- Precision (decimal)
- Unique constraints
- Table names

```
modelBuilder.Entity<Product>()  
    .Property(p => p.UnitPrice)  
    .HasColumnType("decimal(18,2)");
```

- This ensures money values are stored properly.

What Are Migrations?

- Migrations = EF Core's way to generate or update database structure.
- **Why use migrations?**
 - Automatically create tables
 - Apply incremental changes
 - Track database versions
 - Rebuild database anytime
 - No SQL needed

Generating the First Migration

- Run in Package Manager Console:
 - Add-Migration Init
- EF Core will:
 - Scan entities
 - Generate migration file
 - Prepare SQL instructions
- **This does not create the database yet.**

Creating Database from Migration

- To apply changes to SQL Server:
 - Update-Database
- This will:
 - Connect to SQL Server
 - Create database if missing
 - Apply migration
 - Create Product, User tables, etc.

Inventory.Api Project Overview

- Purpose of API:
 - Acts as **middle layer**
 - Receives data from Desktop
 - Validates data
 - Saves to database using DbContext
 - Returns JSON response
- Desktop app never touches the database directly.

NuGet Packages for Inventory.Api

- **Microsoft.EntityFrameworkCore.SqlServer**
 - So API can use DbContext with SQL Server
- **Microsoft.AspNetCore.Identity**
 - Used for PasswordHasher
 - Hashing & verifying passwords
- **Swashbuckle.AspNetCore (auto-added)**
 - Enables Swagger UI
 - Allows testing API visually
- API must reference **Inventory.Data** project.

API Needs to Know Database Connection

- The API must know:
 - Which **database server** to connect to
 - Which **database name** to use
 - Whether to use Windows Authentication or SQL Authentication
 - Additional connection settings
- This information is stored in:
 - **appsettings.json**

What Is appsettings.json?

- The configuration file of your API
- Stores settings such as:
 - Database connection string
 - Logging configuration
 - App-specific settings
 - API keys (in real projects, use secrets file)
- It replaces old web.config (from .NET Framework).

Example appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=.;Database=InventoryDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Understanding the Connection String

- "Server=."
 - Means SQL Server on the same machine
 - Equivalent to (localdb)\\MSSQLLocalDB or localhost
- "Database=InventoryDb"
 - The database to use
 - EF Core will create it on Update-Database
- "Trusted_Connection=True"
 - Uses Windows Authentication
 - No username/password needed
- "MultipleActiveResultSets=true"
 - Allows multiple queries on same connection
 - Required by EF Core

SQL Authentication Version (Optional)

- If you use username/password:
- "DefaultConnection": "Server=.;Database=InventoryDb;UserId=inventory_user;Password=Inventory123!;TrustServerCertificate=True"
- Useful for:
 - Deployment to real server

How the API Reads appsettings.json

- ASP.NET Core automatically loads:
 - appsettings.json
 - appsettings.Development.json
 - environment variables
- `builder.Configuration.GetConnectionString("DefaultConnection")`
 - Reads from JSON
 - Passes to DbContext
 - Connects to SQL Server
- This happens in Program.cs.

Program.cs Overview

- Program.cs is the startup file of the API.
 1. Build the Web Application
 2. Register services:
 - DbContext
 - Controllers
 - PasswordHasher
 3. Load configuration (appsettings.json)
 4. Enable Swagger
 5. Configure middleware (routing, HTTPS redirection)
 6. Start the API

Full Program.cs Example

- `var builder = WebApplication.CreateBuilder(args);`
- `// Read connection string`
- `var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");`
- `// Register DbContext`
- `builder.Services.AddDbContext<AppDbContext>(options =>`
- `options.UseSqlServer(connectionString));`
- `// Add password hasher`
- `builder.Services.AddScoped<IPasswordHasher<AppUser>,
PasswordHasher<AppUser>>();`

Full Program.cs Example

- // Add Controllers and Swagger
- builder.Services.AddControllers();
- builder.Services.AddEndpointsApiExplorer();
- builder.Services.AddSwaggerGen();
- var app = builder.Build();
- if (app.Environment.IsDevelopment())
- {
- app.UseSwagger();
- app.UseSwaggerUI();
- }
- app.UseHttpsRedirection();
- app.UseAuthorization();
- app.MapControllers();
- app.Run();

Why Register DbContext in Program.cs?

- API needs to access the database
- EF Core needs a connection string
- You want to use DbContext through Dependency Injection (DI)
- This line registers DbContext in DI:

```
builder.Services.AddDbContext<AppDbContext>(options =>  
options.UseSqlServer(connectionString));
```

- So controllers can request it automatically in constructors.

What Is Dependency Injection (DI)?

- A modern way to manage dependencies.
- Instead of: `var db = new ApplicationDbContext();`
- `public ProductsController(ApplicationDbContext context)`
- ASP.NET Core **creates the DbContext for us** and injects it.
- Benefits:
 - Cleaner code
 - Easier testing
 - Automatic creation and disposal
 - Fewer errors

Adding Services in Program.cs

- API needs various services:
 - DbContext
 - Controllers
 - PasswordHasher
 - Swagger
- Each registration tells the API: “Make this feature available.”

```
builder.Services.AddControllers();
```

- API now supports controller classes.

What Are Controllers?

- Controllers:
 - Handle HTTP requests
 - Contain actions like GET, POST, PUT, DELETE
 - Communicate with DbContext
 - Return JSON to the Desktop App
- E.g.
 - [ApiController]
 - [Route("api/[controller]")]
 - `public class ProductsController : ControllerBase`

What Are Actions?

- An **action** is a method that responds to an HTTP request.
- Examples:
 - GET /api/products
 - POST /api/products
 - PUT /api/products/5
 - DELETE /api/products/5

```
[HttpGet]  
public async Task<IEnumerable<Product>> GetProducts()  
{ ... }
```

Adding Inventory.Data to Inventory.Api

- To use entities and DbContext inside API:

1. Right-click **Inventory.Api**
2. Select **Add** → **Project Reference**
3. Choose **Inventory.Data**
4. Click OK

- Now you can access:

- AppDbContext
- Product
- AppUser
- EF Core entities

Why We Use DTOs or View Models

- API should separate:
 - Internal database entities
 - Data returned to client
- Example:
 - Don't send PasswordHash to Desktop App
 - Don't allow Desktop to modify DB fields directly
- DTO = Data Transfer Object

Used to shape what API sends or receives.

Example: RegisterRequest DTO

- Desktop sends this to API when registering.
- API hash password and saves an AppUser.

```
public class RegisterRequest
{
    public string UserName { get; set; }
    public string Password { get; set; }
}
```


Example: AuthResponse DTO

- API returns:
 - Success = true/false
 - Friendly message for user
- Useful for Desktop UI notifications.

```
public class AuthResponse
{
    public bool Success { get; set; }
    public string Message { get; set; }
}
```

Register User Workflow

- 1. Desktop App → API

Sends:

```
{  
  "userName": "student1",  
  "password": "Password123!"  
}
```

- API:

- Hashes password
- Saves AppUser to database
- Returns success message

- Desktop:

- Shows “Registration successful”

Login Workflow

- Desktop App sends:

```
{  
  "userName": "student1",  
  "password": "Password123!"  
}
```

- API:
 - Fetches user
 - Verifies password hash
 - Returns success/failure
- Desktop updates UI based on result

Change Password Workflow

- Desktop sends:

```
{  
  "userName": "student1",  
  "oldPassword": "Password123!",  
  "newPassword": "Abc1234!"  
}
```

- API:

- Verifies old password
- Hashes new password
- Saves update

- Desktop displays:

- “Password updated successfully.”

AuthController Overview

- This controller handles everything related to authentication:
 - Register
 - Login
 - Change password
- It uses:
 - AppDbContext (database)
 - PasswordHasher<AppUser> (hashing)
 - DTO classes for input/output
- This demonstrates API + EF Core + security best practices.

AuthController Definition

- Controller receives DbContext via DI
- Hasher used for secure passwords
- Route will be /api/auth

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly IPasswordHasher<AppUser> _hasher;

    public AuthController(AppDbContext context, IPasswordHasher<AppUser> hasher)
    {
        _context = context;
        _hasher = hasher;
    }
}
```

Register Endpoint (POST)

- Key points:
 - Username stored
 - Password hashed
 - No plain-text passwords
 - EF inserts table row

```
[HttpPost("register")]
public async Task<ActionResult> Register(RegisterRequest request)
{
    var user = new AppUser { UserName = request.UserName };
    user.PasswordHash = _hasher.HashPassword(user, request.Password);

    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    return Ok(new AuthResponse { Success = true, Message = "Registered!" });
}
```

Login Endpoint (POST)

- Key points:
 - Finds user
 - Verifies password
 - Returns success message

```
[HttpPost("login")]
public async Task<ActionResult> Login(LoginRequest request)
{
    var user = await _context.Users
        .FirstOrDefaultAsync(x => x.UserName == request.UserName);

    if (user == null) return Unauthorized();

    var result = _hasher.VerifyHashedPassword(user, user.PasswordHash,
        request.Password);

    if (result == PasswordVerificationResult.Failed)
        return Unauthorized();

    return Ok(new AuthResponse { Success = true, Message = "Login successful" });
}
```


Change Password Endpoint (POST)

- Search user by Username
- Verify password
- Hash the new password and save it to database

```
[HttpPost("change-password")]
public async Task<ActionResult> ChangePassword(ChangePasswordRequest req)
{
    var user = await _context.Users.FirstOrDefaultAsync(u => u.UserName == req.UserName);
    if (user == null) return NotFound();

    var verified = _hasher.VerifyHashedPassword(user, user.PasswordHash, req.OldPassword);
    if (verified == PasswordVerificationResult.Failed)
        return BadRequest("Old password incorrect");

    user.PasswordHash = _hasher.HashPassword(user, req.NewPassword);
    await _context.SaveChangesAsync();

    return Ok("Password changed");
}
```

Testing Auth Endpoints with Swagger

- Swagger lets you test:
 - Register new user
 - Login
 - Change password
- Run the API
- E.g. <https://localhost:5000/swagger/index.html>

ProductController Overview

- This controller exposes CRUD operations:
 - GET all products
 - GET by ID
 - POST (Create)
 - PUT (Update)
 - DELETE (Remove)
- Uses:
 - AppDbContext
 - EF Core tracking
 - JSON models

GET All Products

- This will get all list of products from Product table.

```
[HttpGet]  
public async Task<IEnumerable<Product>> GetAll()  
{  
    return await _context.Products.ToListAsync();  
}
```

GET Product by Id

- This will get Product by productId

```
[HttpGet("{id:int}")]
public async Task<ActionResult<Product>> Get(int id)
{
    var p = await _context.Products.FindAsync(id);
    if (p == null) return NotFound();
    return Ok(p);
}
```

Create Product (POST)

- Accept Product
- Add product too the context
- Save the product to database

```
[HttpPost]
public async Task<ActionResult<Product>> Create(Product product)
{
    _context.Products.Add(product);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(Get), new { id = product.Id }, product);
}
```

Update Product (PUT)

```
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, Product product)
{
    if (id != product.Id) return BadRequest();

    _context.Entry(product).State = EntityState.Modified;
    await _context.SaveChangesAsync();
    return NoContent();
}
```

Delete Product (DELETE)

```
[HttpDelete("{id}")]
public async Task<ActionResult> Delete(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product == null) return NotFound();

    _context.Products.Remove(product);
    await _context.SaveChangesAsync();
    return NoContent();
}
```


Summary of API Responsibilities

- API handles:
 - User authentication
 - Product CRUD
 - Validation
 - Database access via EF Core
 - Returns JSON
 - Acts as middle layer

Desktop Project Overview

- Desktop (WinForms) responsibilities:
 - Display UI
 - Button events
 - Use HttpClient to call API
 - Show data in ListView
 - Send JSON, receive JSON
- No SQL code, no direct DB connection.

Adding HttpClient to Desktop App

- Create a class:
- **Name: ApiClient.cs**
- Purpose; Handles communication with API.
- Use:
 - Separation of concerns
 - Reusable service class
 - Modern coding style

ApiClient (Base Setup)

```
public class ApiClient
{
    private readonly HttpClient _http;

    public ApiClient()
    {
        _http = new HttpClient();
        _http.BaseAddress = new Uri("https://localhost:7296");
    }
}
```

ApiClient: Get Products

```
public async Task<List<Product>> GetProductsAsync()
{
    var products =
        await _http.GetFromJsonAsync<List<Product>>("api/products");
    return products ?? new List<Product>();
}
```

ApiClient: Create Product

```
public async Task<Product?> CreateProductAsync(Product p)
{
    var resp = await _http.PostAsJsonAsync("api/products", p);
    resp.EnsureSuccessStatusCode();
    return await resp.Content.ReadFromJsonAsync<Product>();
}
```

ApiClient: Login

```
public async Task<AuthResponse?> LoginAsync(string user, string pass)
{
    var request = new { userName = user, password = pass };
    var resp = await _http.PostAsJsonAsync("api/auth/login", request);
    return await resp.Content.ReadFromJsonAsync<AuthResponse>();
}
```

Using ApiClient in Forms

```
private async void btnLoad_Click(object sender, EventArgs e)
{
    var api = new ApiClient();
    var products = await api.GetProductsAsync();
    dgvProducts.DataSource = products;
}
```


What Is LINQ?

- LINQ stands for: **Language-Integrated Query**
- It allows querying:
 - Objects
 - Collections
 - Lists
 - Databases (via EF Core)
 - XML
 - JSON
- Using **C# syntax**, instead of SQL or loops.

Why LINQ Matters

- LINQ is powerful because:
 - Replaces loops with clean expressions
 - Works with lists, arrays, DbSet, JSON
 - Integrates smoothly with EF Core
 - Avoids manual SQL writing
 - Enables functional programming concepts
 - Makes code readable and shorter

Traditional Way vs LINQ

- Traditional:

```
List<Product> results = new List<Product>();  
foreach (var p in products)  
{  
    if (p.UnitPrice > 100)  
        results.Add(p);  
}
```

- LINQ:

```
var results = products.Where(p => p.UnitPrice > 100).ToList();
```

LINQ Query Syntax vs Method Syntax

- Query Syntax:

```
var q = from p in products
        where p.UnitPrice > 100
        select p;
```

- Method Syntax (most popular):

```
var q = products.Where(p => p.UnitPrice > 100);
```

- EF Core internally uses **method syntax**.

Important LINQ Methods

- Most commonly used:
 - Where()
 - Select()
 - OrderBy()
 - OrderByDescending()
 - FirstOrDefault()
 - SingleOrDefault()
 - Any()
 - Take()
 - Skip()
 - ToList()

Using Where()

- Filters items based on a condition.

```
var expensive = products  
    .Where(p => p.UnitPrice > 100)  
    .ToList();
```

- Equivalent to SQL:

```
SELECT * FROM Products WHERE UnitPrice > 100;
```

Using Select()

- Selects specific fields or transforms data.

```
var names = products  
    .Select(p => p.Name)  
    .ToList();
```

- Or create new objects:

```
var info = products.Select(p => new {  
    p.Name,  
    p.UnitPrice  
}).ToList();
```

Using OrderBy()

- Ascending:

```
var sorted = products  
    .OrderBy(p => p.UnitPrice)  
    .ToList();
```

- Descending:

```
sorted = products  
    .OrderByDescending(p => p.UnitPrice)  
    .ToList();
```


FirstOrDefault()

- Gets the first item or returns null.

```
var product = products  
    .FirstOrDefault(p => p.Name == "Laptop");
```

- Used frequently with EF Core:

```
var user = _context.Users  
    .FirstOrDefault(u => u.UserName == request.UserName);
```

SingleOrDefault()

- Ensures only **one** item exists.

```
var p = products.SingleOrDefault(p => p.Id == 3);
```

- Throws exception if more than one exists.

Any()

- Checks if something exists.

```
bool hasCheapItems = products.Any(p => p.UnitPrice < 50);
```

- Very useful for:
 - Validation
 - Existence check
 - Authentication logic

Take() and Skip() (Paging)

- Take 5

```
var top5 = products.Take(5).ToList();
```

- Skip 10

```
var next = products.Skip(10).Take(5).ToList();
```

- Used in paging APIs.

Combining LINQ Methods

- Chaining operations = clean logic.

```
var filtered = products
    .Where(p => p.UnitPrice >= 100)
    .OrderBy(p => p.Name)
    .Select(p => new { p.Name, p.UnitPrice })
    .ToList();
```

LINQ with EF Core

- EF Core converts LINQ into SQL.

```
var items = await _context.Products  
    .Where(p => p.QuantityInStock < 10)  
    .OrderBy(p => p.Name)  
    .ToListAsync();
```

- EF generates SQL like:

```
SELECT * FROM Products  
WHERE QuantityInStock < 10  
ORDER BY Name ASC;
```

Example in Controller

```
[HttpGet("low-stock")]
public async Task<IEnumerable<Product>> GetLowStock()
{
    return await _context.Products
        .Where(p => p.QuantityInStock < 5)
        .ToListAsync();
}
```

LINQ GroupBy

- Useful for:
 - Reporting
 - Charts
 - Statistics in dashboard

```
var groups = products
    .GroupBy(p => p.Category)
    .Select(g => new { Category = g.Key, Count =
g.Count() })
    .ToList();
```


Thank You!

?